

ENHANCING SYSTEM VERILOG WITH AOP CONCEPT

Mimic AOP-like functionality using OOP techniques in System Verilog

■ NISHITH SHUKLA

Aspect-oriented programming (AOP) is a well-established technique for adding, modifying or enhancing functionality to a class or environment. It has become *de-facto* standard for developing reusable verification environment or component.

This article explains semantics differences between AOP and object-oriented programming (OOP) and, where relevant, presents how similar functionality can be obtained using System Verilog.

Fundamentals

Despite having many commonalities, 'e' and System Verilog programming languages differ significantly in certain critical areas. For instance, 'e' supports AOP techniques, while System Verilog supports OOP techniques.

AOP is popular because it's very easy to maintain the scattered code. Programmers can easily express cross-cutting concerns in standalone modules called 'Aspect.' OOP doesn't allow you to decompose a problem into all of its concerns. You can only encapsulate some of the concerns. It really doesn't allow you to deal with cross-cutting concerns. Making OOP do something which it doesn't naturally do, needs some extra efforts and thinking.

It's understandable that the ultimate difference between AOP and OOP comes down to organisation of the code. In AOP the code is organised by functionality, while in OOP the code is organised by classes and objects. The

easiest way to understand this concept is to take a packet written in 'e' and then see how it would be written in System Verilog:

Sample 'e' code:

```
<'
type packetType : [DBTRANS];
struct TransBaseT {
    isFirstTrans : bool;
    kind : packetType;
    keep isFirstTrans == false;

    processTrans() is {
    }
    print_me() is {
        message(LOW, "-----");
        message(LOW, "Packet Information");
        message(LOW, "-----");
    }
}
>

<'
extend DBTRANS TransBaseT {
    %addr : uint(bits:32);
    %data : uint(bits:32);
    %wr_rd : bool;
    keep addr in [0x0000..0xffff];
    keep wr_rd = select { 50: true,
                        50: false
    }
    print_me() is also {
        message(LOW, "Addr= ", addr, "Data=",
            data, "WR_RD=", wr_rd);
    }
    processTrans() is only {
        message(LOW, "Processing Databus
            Transaction");
        print_me();
    }
}
>
```

Equivalent System Verilog code:

```
class TransBaseT;
    rand bool isFirstTrans;
    constraint C1 {
        isFirstTrans == false;
    }
    virtual function processTrans();
    endfunction
    virtual function printMe();
    endfunction
endclass

class DbTransT extends TransBaseT;
    rand bit [31:0] addr;
    rand bit [31:0] data[];
    rand bool wr_rd;
    constraint C2 { addr inside
        {32'h0:32'hffff} ;
        data.size() inside
        {32'h0:32'hff}
    }
    constraint C3 { wr_rd dist { 50 :
        true, 50 : false }
    }
    virtual function printMe();
        $display("DbTransT : Addr= %h
        Data = %h Wr_Rd=%h",
            addr, data, wr_rd);
    endfunction
    virtual function processTrans();
        $display("Processing DbTransT
        Packet");
        printMe();
    endfunction
endclass
```

In classic OOP, you should instantiate the extended class not the base class. If you extend a class, the base class and classes which have already been instantiated will not see changes. Nor would you want them to because the risk is very high that one would carelessly extend a class and corrupt the functionality of all instantiated classes.